EMPLOYEE DATABASE ANALYSIS USING SQL

1. Introduction

This project focuses on analyzing an employee database using SQL. The HR table contains details such as employee ID, name, salary, department, and hiring date.

The purpose of this project is to demonstrate SQL operations like data retrieval, filtering, aggregation, grouping, and advanced analytics using window functions.

SQL plays a major role in data analysis, making it easy to extract insights for decision-making in organizations.

2. Database Creation

Query:

```
create database e;

use e;
```

Explanation:

The above queries are used to create a new database named 'e' and select it for further operations. This ensures all tables and queries are executed within this database.

3. Viewing Data

Query:

```
select * from HR;
```

Explanation:

This query retrieves all columns and rows from the HR table. It is useful for checking the structure and contents of the dataset.

4. Selecting Specific Columns

Query:

```
select Employee_ID, Full_Name, Salary_INR from HR;
```

Explanation:

This query selects only specific columns instead of displaying the entire table, improving performance and readability.

5. Filtering Data using WHERE

Query:

```
select Full_Name, Salary_INR from hr where Salary_INR > 50000;

select Full_Name, Department from HR where Department = 'HR';
```

Explanation:

The WHERE clause filters records based on conditions. It helps in extracting only relevant data such as employees with high salary or specific departments.

6. Sorting Data

Query:

```
select Full_Name, Salary_INR from hr order by Salary_INR desc;
```

Explanation:

The ORDER BY clause sorts the data. 'DESC' sorts in descending order, showing highest salaries first.

7. Aggregate Functions

Queries:

```
select count(Full_Name) from hr;

select sum(Salary_INR) from hr;

select min(Salary_INR) from hr;

select max(Salary_INR) from hr;

select avg(Salary_INR) from hr;
```

Explanation:

Aggregate functions perform calculations on multiple rows. COUNT gives number of employees, SUM gives total salary, MIN and MAX give lowest and highest salaries, and AVG calculates average salary.

8. DISTINCT Values

Query:

select distinct(Department) from hr;

Explanation:

This query returns unique department names by removing duplicate values.

9. GROUP BY Clause

Queries:

select sum(Salary_INR), Department from hr group by Department;

select Department, count(*) from hr group by Department;

Explanation:

GROUP BY groups rows with similar values. It is used to perform calculations like total salary and employee count for each department.

10. LIKE Operator

Query:

select Full_Name from hr where Full_Name like 'a%';

Explanation:

The LIKE operator is used for pattern matching. 'a%' retrieves all names starting with 'A'.

11. BETWEEN Operator

Query:

select Full_Name, Salary_INR from hr where Salary_INR BETWEEN 100000 AND 700000;

Explanation:

BETWEEN filters values within a range, including both lower and upper limits.

12. CASE Statement

Query:

```
select Full_Name, Salary_INR,
CASE
   when Salary_INR >= 700000 then "high"
   when Salary_INR >= 500000 then "medium"
   when Salary_INR >= 100000 then "low"
end as "priority"
from hr;
```

Explanation:

CASE statement is used for conditional logic. It categorizes employees based on salary into high, medium, and low groups.

13. LIMIT Clause

Queries:

```
select Full_Name, Salary_INR from hr order by Salary_INR DESC limit 5;
```

```
select Full_Name, Salary_INR from hr order by Salary_INR DESC limit 2;
```

Explanation:

The LIMIT clause restricts the number of rows returned, useful for finding top records.

14. Date Filtering

Query:

```
select Full_Name, Hire_Date from hr where Hire_Date >= '2022-01-01';
```

Explanation:

This query retrieves employees hired after a specific date, helping analyze recent recruitment.

15. Subquery

Query:

```
select Full_Name, Salary_INR from hr where Salary_INR > (select avg(Salary_INR) from hr);
```

Explanation:

A subquery is a query inside another query. It helps compare values dynamically, such as finding employees earning above average salary.

16. Finding Duplicates

Query:

```
SELECT Full_Name, COUNT(*) AS count
FROM hr
GROUP BY Full_Name
HAVING COUNT(*) > 1;
```

Explanation:

This query identifies duplicate names in the table using GROUP BY and HAVING.

17. Window Functions

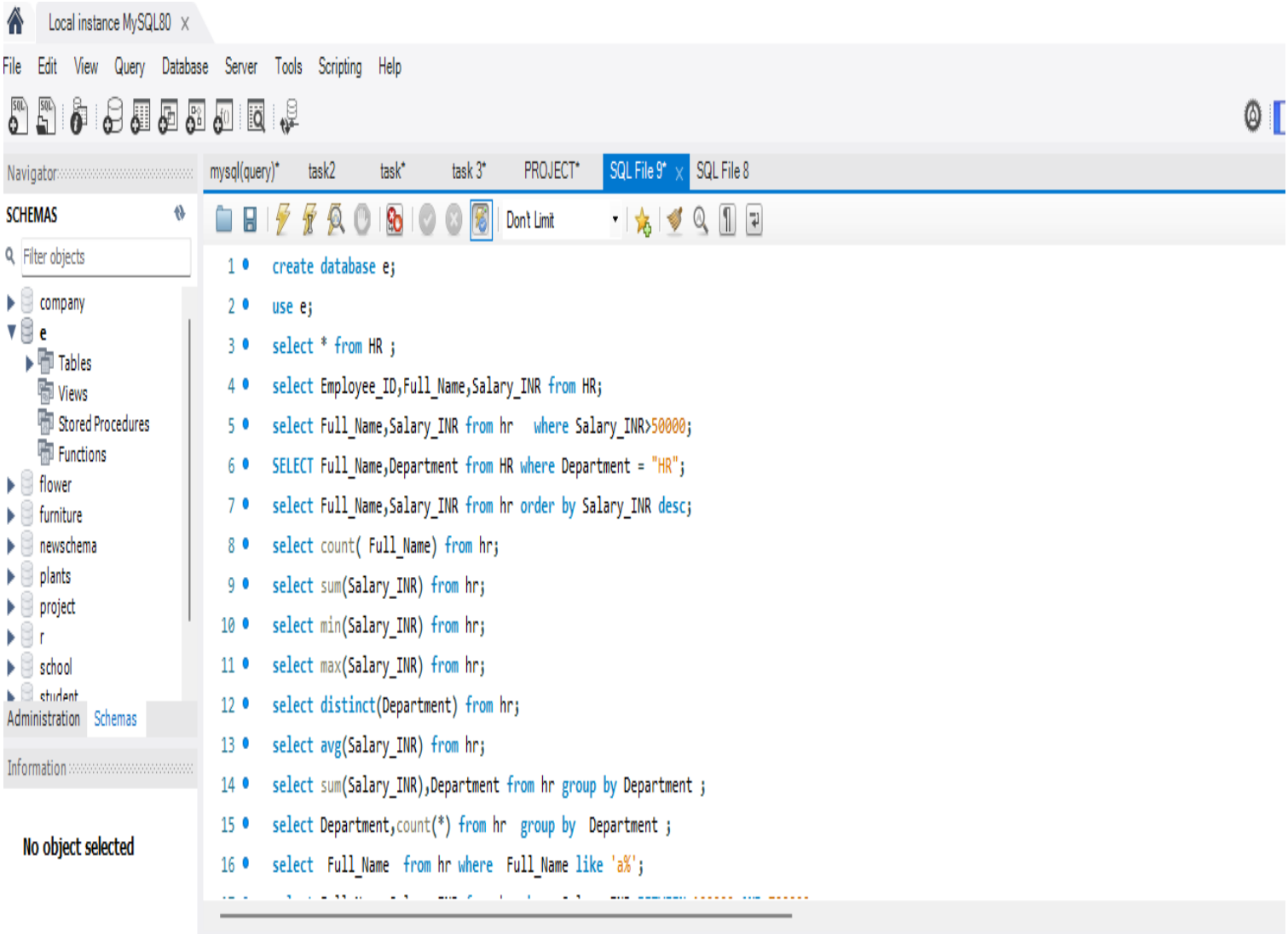
Queries:

```
select Full_Name, Salary_INR,
Dense_rank() over (order by Salary_INR desc) as rankwise
from hr;
```

```
select Full_Name,
sum(Salary_INR) over (order by Salary_INR desc) as salarywise
from hr;
```

Explanation:

Window functions perform calculations across rows without grouping. Dense_rank assigns ranks, while SUM over calculates cumulative totals.



18. Conclusion

This project demonstrates how SQL can be used to efficiently analyze employee data.

It covers basic to advanced SQL concepts such as filtering, grouping, subqueries, and window functions.

These techniques are widely used in real-world data analytics and business intelligence applications.